LangChain: Building Intelligent LLM-Powered Applications from the Ground Up

> *A deep technical guide covering architecture, components, working code, and real-world use cases.*

---

1. Introduction to LangChain

Large Language Models like GPT-4 and Claude are powerful in isolation but building a production-ready AI application requires far more than a single API call. You need to manage conversation history, inject external data, connect tools, and orchestrate multi-step reasoning. Doing all of this from scratch is error-prone and repetitive.

LangChain is an open-source Python framework that solves exactly this. It provides a modular, composable architecture for building LLM-powered applications by abstracting the hard parts: prompt management, chaining multiple LLM calls, connecting to external tools, managing memory, and retrieving documents from vector databases.

Why LangChain Matters

Without LangChain, you'd manually handle prompt formatting, context injection, tool routing, and state management for every project. LangChain standardizes these patterns into reusable components so you can focus on application logic instead of infrastructure.

It solves three critical problems in modern LLM development:

- **Orchestration** — Coordinating multiple LLM calls in a defined sequence
- **Chaining** — Passing outputs of one step as inputs to the next automatically
- **Tool Integration** — Giving LLMs the ability to call search engines, databases, APIs, and custom Python functions

---

2. Core Components of LangChain

LangChain's power comes from the way its components snap together. Each one is independently useful, but they become exponentially more powerful when combined.

2.1 LLMs and Chat Models

What it is: The foundational abstraction over any language model — whether OpenAI's GPT, HuggingFace models, or locally running Ollama models. LangChain wraps them behind a unified interface so you can swap providers without rewriting logic.

Why it exists: Prevents vendor lock-in and allows seamless provider switching.

```
from langchain_openai import ChatOpenAI
```

```
# Initialize the model (uses OPENAI_API_KEY env variable)
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# Direct invocation
response = llm.invoke("Explain transformers in two sentences.")
print(response.content)
```

> **HuggingFace Alternative (no OpenAI key needed):**

```
from langchain_huggingface import HuggingFaceEndpoint

llm = HuggingFaceEndpoint(
    repo_id="mistralai/Mistral-7B-Instruct-v0.2",
    temperature=0.5
)
response = llm.invoke("What is attention mechanism?")
print(response)
```

2.2 Prompts and Prompt Templates

What it is: A structured way to build dynamic prompts by injecting variables into a template at runtime rather than hardcoding strings.

Why it exists: Hardcoded prompts break the moment requirements change. Templates make prompts reusable, testable, and maintainable.

```
from langchain_core.prompts import ChatPromptTemplate

# Define a reusable template with placeholders
template = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {domain}. Answer concisely."),
    ("human", "{question}")
])

# Format it with actual values
formatted = template.format_messages(
    domain="machine learning",
    question="What is gradient descent?"
)

# Pass to LLM
response = llm.invoke(formatted)
print(response.content)
```

2.3 Chains

What it is: A chain connects components sequentially typically a prompt template flows into an LLM, and its output is passed to a parser or the next chain. LangChain uses the **LCEL (LangChain Expression Language)** | pipe syntax.

Why it exists: Manual output passing between steps is brittle. Chains enforce structured flow and make pipelines readable.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo")

# Build a chain using LCEL pipe syntax
chain = (
    ChatPromptTemplate.from_template("Summarize this text in one sentence:
{text}")
    | llm
    | StrOutputParser()
)

# Run it
result = chain.invoke({"text": "LangChain is a framework for building LLM
applications..."})
print(result)
```

2.4 Memory

What it is: Memory allows a conversational chain to retain information across multiple turns. Without it, every LLM call is stateless the model forgets the previous message immediately.

Why it exists: A chatbot that forgets context after every message is useless for real conversations.

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory

llm = ChatOpenAI(model="gpt-3.5-turbo")

# Prompt template with a placeholder for chat history
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}")
])

chain = prompt | llm
```

```
# In-memory store for session histories
store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

# Wrap chain with memory management
chain_with_memory = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="history"
)

# Turn 1
r1 = chain_with_memory.invoke(
    {"input": "My name is Shravan."},
    config={"configurable": {"session_id": "session_1"}}
)
print(r1.content)

# Turn 2 — model should remember the name
r2 = chain_with_memory.invoke(
    {"input": "What is my name?"},
    config={"configurable": {"session_id": "session_1"}}
)
print(r2.content)
```

2.5 Agents

What it is: An agent is a reasoning loop where the LLM decides *which tool to call*, calls it, observes the result, and repeats until it can answer the question. This is based on the **ReAct (Reasoning + Acting)** framework.

Why it exists: Static chains follow fixed paths. Agents make dynamic decisions based on the query, enabling open-ended problem solving.

```
from langchain_openai import ChatOpenAI
from langchain_community.tools import DuckDuckGoSearchRun
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
search_tool = DuckDuckGoSearchRun()

# Pull a standard ReAct prompt from LangChain hub
prompt = hub.pull("hwchase17/react")

# Create agent
```

```
agent = create_react_agent(llm, tools=[search_tool], prompt=prompt)
executor = AgentExecutor(agent=agent, tools=[search_tool], verbose=True)

result = executor.invoke({"input": "What is the latest version of LangChain?"})
print(result["output"])
```

2.6 Tools

What it is: Tools are functions (Python callables) that agents can invoke. They wrap APIs, calculators, databases, code executors, or any custom logic.

Why it exists: LLMs can reason and generate text, but they cannot execute code, browse the web, or query databases on their own. Tools bridge that gap.

```
from langchain_core.tools import tool

@tool
def calculate_area(length: float, width: float) -> float:
    """Calculate the area of a rectangle given its length and width."""
    return length * width

# The agent can now decide to call this function when needed
print(calculate_area.invoke({"length": 5.0, "width": 3.0}))
# Output: 15.0
```

2.7 Document Loaders

What it is: Document loaders import text from external sources PDFs, web pages, CSVs, Notion pages, YouTube transcripts and convert them into LangChain `Document` objects.

Why it exists: LLMs cannot directly read your files. Loaders standardize how raw content is ingested into the pipeline.

```
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader

# Load a PDF
pdf_loader = PyPDFLoader("report.pdf")
pdf_docs = pdf_loader.load()

# Load a webpage
web_loader = WebBaseLoader("https://python.langchain.com/docs/introduction/")
web_docs = web_loader.load()

print(f"Loaded {len(pdf_docs)} pages from PDF")
print(f"First 200 chars: {web_docs[0].page_content[:200]}")
```

2.8 Indexes and Vector Stores

What it is: Documents are split into chunks, converted into numerical embeddings (vectors), and stored in a vector database. At query time, the most semantically similar chunks are retrieved and fed into the LLM as context. This is the foundation of **RAG (Retrieval-Augmented Generation)**.

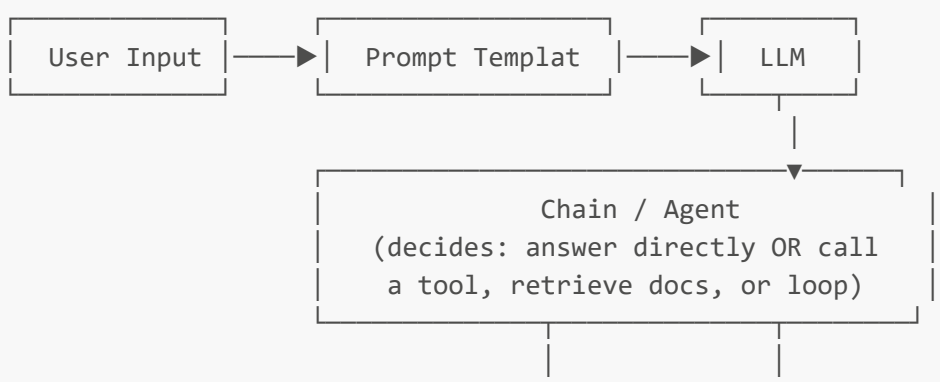
Why it exists: LLMs have token limits and cannot absorb entire books. Vector search retrieves only the relevant excerpts.

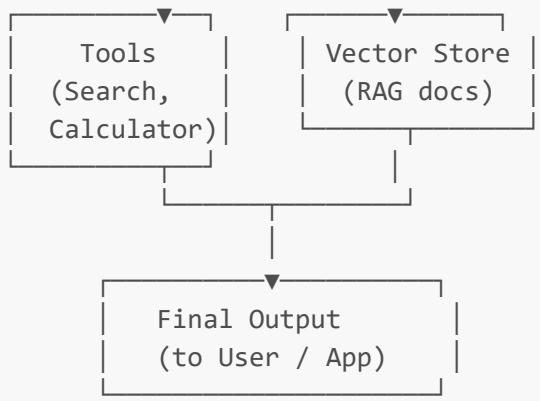
```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Sample documents
from langchain_core.documents import Document

docs = [
    Document(page_content="LangChain is a framework for LLM applications."),
    Document(page_content="FAISS is a library for efficient similarity search."),
    Document(page_content="Transformers are the backbone of modern NLP models."),
]

# Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=10)
chunks = splitter.split_documents(docs)

# Create embeddings and store in FAISS
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(chunks, embeddings)

# Semantic search
results = vectorstore.similarity_search("What is LangChain?", k=2)
for r in results:
    print(r.page_content)
```

---

3. Architecture

The full LangChain execution flow follows this path from user query to final output:





Step-by-step flow:

1. User sends a query to the application.
2. A `PromptTemplate` formats the query with context, system instructions, or retrieved documents.
3. The formatted prompt is sent to the **LLM**.
4. If a simple `Chain` is used, the output is parsed and returned directly.
5. If an **Agent** is used, the LLM decides whether to call a tool or retrieve documents, then loops until it has a final answer.
6. **Memory** injects prior conversation turns into the prompt at each step.
7. The final output is returned to the user or downstream system.

---

4. Hands-on Code Examples

Complete RAG Pipeline (Document Q&A)

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

# ── Step 1: Prepare documents ──────────────────────────────────────
documents = [
    Document(page_content="LangChain was created by Harrison Chase in 2022."),
    Document(page_content="LangChain supports OpenAI, HuggingFace, Anthropic, and
Cohere."),
    Document(page_content="LCEL is LangChain Expression Language using the pipe
operator."),
    Document(page_content="Agents use ReAct prompting to decide tool usage
dynamically."),
]

# ── Step 2: Split and embed ────────────────────────────────────────
splitter = RecursiveCharacterTextSplitter(chunk_size=150, chunk_overlap=20)
chunks = splitter.split_documents(documents)
```

```
vectorstore = FAISS.from_documents(chunks, OpenAIEmbeddings())
retriever = vectorstore.as_retriever(search_kwargs={"k": 2})

# ── Step 3: Build RAG chain ──────────────────────────────────────
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

prompt = ChatPromptTemplate.from_template("""
Answer the question using ONLY the context below. If unsure, say "I don't know."

Context:
{context}

Question: {question}
""")

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

# ── Step 4: Query ────────────────────────────────────────────────
answer = rag_chain.invoke("Who created LangChain?")
print(answer)
# Output: LangChain was created by Harrison Chase in 2022.
```

---

5. Real-World Use Cases

Use Case 1 — Customer Support Chatbot with Memory

Problem: A SaaS company needs a support bot that understands previous messages in the same session and can answer questions about their product documentation.

Solution: Load product docs into a FAISS vector store (RAG). Wrap the chain with `RunnableWithMessageHistory` to maintain per-session memory. On each query, retrieve relevant doc chunks and include prior conversation turns in the prompt.

Components used: `ChatOpenAI`, `FAISS`, `OpenAIEmbeddings`, `RunnableWithMessageHistory`, `ChatPromptTemplate`, `PyPDFLoader`

Use Case 2 — Research Assistant Agent with Web Search

Problem: A researcher wants to ask open-ended questions that require browsing the web, summarizing articles, and synthesizing information from multiple sources.

Solution: Build a ReAct agent equipped with `DuckDuckGoSearchRun` and a custom `summarize_url` tool. The agent autonomously decides when to search, what to fetch, and how to synthesize the answer.

Components used: `ChatOpenAI`, `DuckDuckGoSearchRun`, `create_react_agent`, `AgentExecutor`, `@tool`

Use Case 3 — Automated Code Review Pipeline

Problem: A development team wants to automatically review pull request code for bugs, security vulnerabilities, and style issues before human review.

Solution: A sequential chain first extracts the diff from a GitHub webhook, then passes it through a `ChatPromptTemplate` instructing the LLM to act as a senior code reviewer. A second chain generates a structured Markdown report. Output is posted back to the PR as a comment via a GitHub tool.

Components used: `ChatOpenAI`, `ChatPromptTemplate`, LCEL chain, `@tool` (GitHub API wrapper), `StrOutputParser`

---

6. Advantages and Limitations

Strengths

Modularity is LangChain's defining trait every component (LLM, memory, retriever, tool) is independently replaceable. Swapping GPT-4 for Mistral, or FAISS for Pinecone, requires changing a single line.

Rapid prototyping is dramatically faster compared to building from scratch. A working RAG pipeline can be assembled in under 50 lines of code.

Ecosystem integrations span over 100 LLM providers, 50+ vector stores, and dozens of document loaders — all behind unified interfaces.

LCEL makes complex pipelines readable and composable using intuitive `|` pipe syntax, similar to Unix pipelines.

Limitations

Latency overhead is real — the abstraction layers add processing time, and multi-step agent loops can become slow for time-sensitive applications.

Debugging complexity increases with chain depth. When something fails in a 5-step chain, isolating the exact failure point requires careful logging.

API costs accumulate quickly with agents that loop many times or chains that embed large document sets.

Frequent API changes have been a historical pain point LangChain has gone through multiple breaking refactors (v0.1 → v0.2 → v0.3), requiring constant dependency management.

When NOT to Use LangChain

Do not use LangChain when your task is a single, direct LLM call with no orchestration needed the added abstraction is pure overhead. Similarly, for production systems with extreme latency requirements, a lean

custom implementation will outperform the framework. If your team lacks experience debugging abstracted pipelines, simple direct API usage may be more maintainable.

---

7. Conclusion

Key Takeaways

LangChain transforms LLMs from isolated text generators into reasoning engines that can remember context, retrieve knowledge, execute tools, and chain complex multi-step logic. Its modular design means you learn once and apply everywhere swap providers, scale components, and iterate rapidly.

What This Assignment Taught

Working through LangChain's components reveals a consistent design philosophy: separate *what to do* (your application logic) from *how to do it* (the underlying model and infrastructure). LCEL's pipe syntax makes this separation visual and explicit. The hardest but most valuable insight is understanding agents once you see how a ReAct loop works internally, a whole category of autonomous AI applications becomes buildable.

Future Scope

LangGraph extends LangChain's agent model to support cyclic, stateful workflows allowing agents to loop, branch, and maintain complex state across many steps. This is the direction the ecosystem is moving toward for production multi-agent systems.

Multi-agent architectures, where specialized agents collaborate (one browses, one reasons, one writes), represent the next frontier and are already being built using LangGraph as the coordination layer.

The gap between a developer who *uses* LangChain and one who *designs systems with it* is understanding the internals covered in this blog. Master the components, and you can architect any LLM application.

Save this file , like , share , comment and follow for more such content

#BuildInPublic #DevsDontLie #DevDiscipline #LangChain #AIForEveryone #InnomaticsResearchLabs

Not Done Yet !

With Love , Practice and Dedication - Shravan Shidruk

LinkedIn: Shravan Shidruk

GitHub: Shravan Shidruk
